

Pertemuan 6 Searching dan Sorting

Pemrograman untuk Ilmu Data dan Komputasi

MA25-21008 Kelas RA dan RB

Rifky Fauzi

Triyana Muliawati, S.Si., M.Si.

Program Studi Matematika
Institut Teknologi Sumatera

Semester Ganjil 2026/2027

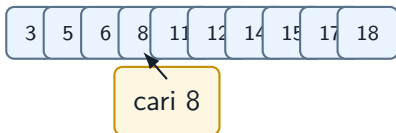
Hari ini

- Membedakan mencari data dan mengurutkan data.
- Menelusuri sequential search dan binary search.
- Menelusuri selection sort, bubble sort, dan merge sort.
- Menghitung banyak kerja $T(n)$ dari proses yang terlihat.
- Mengubah $T(n)$ menjadi orde pertumbuhan.

Dua pekerjaan dasar pada data

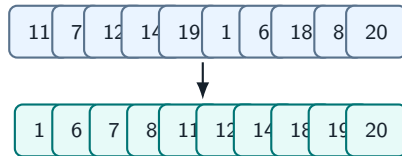
Searching

Diberikan kumpulan data dan satu target. Pertanyaannya: target ada di mana, atau apakah target tidak ada?



Sorting

Diberikan kumpulan data. Pertanyaannya: bagaimana menyusun data agar urut?



Cara membaca algoritma hari ini

Tentukan ukuran input n , pilih operasi yang dihitung, formulasikan $T(n)$, lalu ambil orde dominannya.

Searching I: sequential search

Tujuan

Cari target dalam list. Algoritma memeriksa elemen dari kiri ke kanan sampai ketemu atau sampai list habis.

```
def sequential_search(a, target):  
    for i in range(len(a)):  
        if a[i] == target:  
            return i  
    return -1
```

Banyak kerja: perbandingan

Misalkan panjang list adalah n .

posisi target	contoh	$T(n)$
awal	a_0	1
tengah	$a_{\lfloor n/2 \rfloor}$	$\lfloor n/2 \rfloor + 1$
akhir	a_{n-1}	n
tidak ada	—	n

Searching I: sequential search

Tujuan

Cari target dalam list. Algoritma memeriksa elemen dari kiri ke kanan sampai ketemu atau sampai list habis.

```
def sequential_search(a, target):  
    for i in range(len(a)):  
        if a[i] == target:  
            return i  
    return -1
```

Banyak kerja: perbandingan

Misalkan panjang list adalah n .

posisi target	contoh	$T(n)$
awal	a_0	1
tengah	$a_{\lfloor n/2 \rfloor}$	$\lfloor n/2 \rfloor + 1$
akhir	a_{n-1}	n
tidak ada	—	n

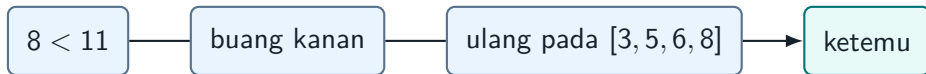
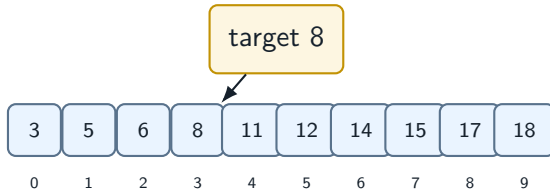
Worst case

Untuk analisis dasar, sequential search biasanya dibaca $O(n)$.

Searching II: binary search pada list terurut

Syarat penting

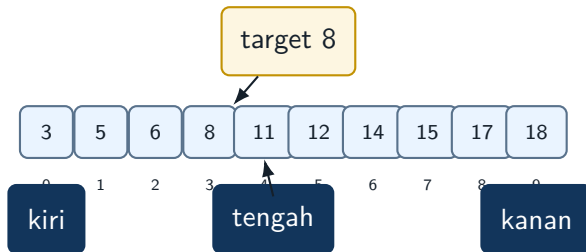
Binary search hanya masuk akal jika data sudah terurut.



Searching II: binary search pada list terurut

Syarat penting

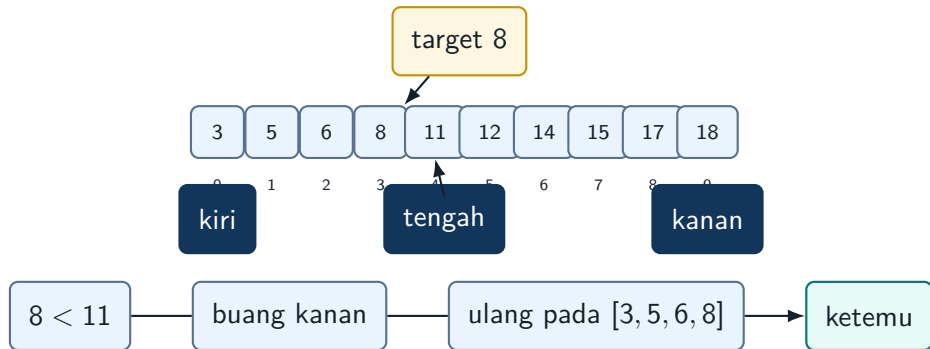
Binary search hanya masuk akal jika data sudah terurut.



Searching II: binary search pada list terurut

Syarat penting

Binary search hanya masuk akal jika data sudah terurut.



Binary search: dari separuh ke $\log n$

Ukuran kandidat setelah k langkah

langkah	sisa kandidat
0	n
1	$n/2$
2	$n/4$
3	$n/8$
k	$n/2^k$

Kapan berhenti?

Kandidat tinggal satu ketika

$$\frac{n}{2^k} \leq 1.$$

Maka

$$n \leq 2^k \Rightarrow k \geq \log_2 n.$$

Binary search: dari separuh ke $\log n$

Ukuran kandidat setelah k langkah

langkah	sisa kandidat
0	n
1	$n/2$
2	$n/4$
3	$n/8$
k	$n/2^k$

Kapan berhenti?

Kandidat tinggal satu ketika

$$\frac{n}{2^k} \leq 1.$$

Maka

$$n \leq 2^k \Rightarrow k \geq \log_2 n.$$

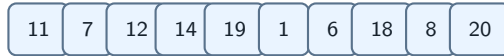
Orde

Binary search membutuhkan sekitar $\log_2 n$ perbandingan pada worst case, sehingga ordonya $O(\log n)$.

Sorting I: selection sort

Ide

Pada setiap pass, cari nilai terkecil pada bagian yang belum terurut, lalu tukar ke posisi pertama bagian tersebut.



semua posisi belum terurut

Makna pass

Pass ke-1 menempatkan elemen terkecil pada posisi final. Pass berikutnya bekerja pada bagian kanan yang tersisa.

Sorting I: selection sort

Ide

Pada setiap pass, cari nilai terkecil pada bagian yang belum terurut, lalu tukar ke posisi pertama bagian tersebut.



semua posisi belum terurut

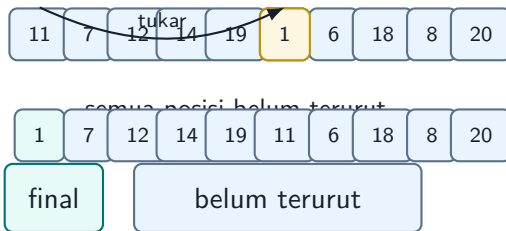
Makna pass

Pass ke-1 menempatkan elemen terkecil pada posisi final. Pass berikutnya bekerja pada bagian kanan yang tersisa.

Sorting I: selection sort

Ide

Pada setiap pass, cari nilai terkecil pada bagian yang belum terurut, lalu tukar ke posisi pertama bagian tersebut.



Makna pass

Pass ke-1 menempatkan elemen terkecil pada posisi final. Pass berikutnya bekerja pada bagian kanan yang tersisa.

Selection sort: menghitung $T(n)$

Jumlah perbandingan

Untuk mencari minimum:

pass	perbandingan
1	$n - 1$
2	$n - 2$
3	$n - 3$
$\vdots$	$\vdots$
$n - 1$	1

Formulasi

$$\begin{aligned}T(n) &= (n - 1) + (n - 2) + \cdots + 1 \\&= 1 + 2 + \cdots + (n - 1) \\&= \frac{n(n - 1)}{2} \\&= \frac{1}{2}n^2 - \frac{1}{2}n.\end{aligned}$$

Selection sort: menghitung $T(n)$

Jumlah perbandingan

Untuk mencari minimum:

pass	perbandingan
1	$n - 1$
2	$n - 2$
3	$n - 3$
$\vdots$	$\vdots$
$n - 1$	1

Formulasi

$$\begin{aligned}T(n) &= (n - 1) + (n - 2) + \cdots + 1 \\&= 1 + 2 + \cdots + (n - 1) \\&= \frac{n(n - 1)}{2} \\&= \frac{1}{2}n^2 - \frac{1}{2}n.\end{aligned}$$

Orde

Suku dominan adalah n^2 , jadi selection sort berada pada $O(n^2)$.

Bagian kiri yang sudah final tidak berubah lagi; dalam analisis algoritma, sifat tetap seperti ini biasa dikenal sebagai *invariant*.

Sorting II: bubble sort sebagai pembandingan

Ide

Bandingkan pasangan yang bersebelahan. Jika urutannya salah, tukar. Setelah satu lintasan, elemen terbesar bergerak ke ujung kanan.



Banyak kerja

Worst case juga membentuk deret

$$(n - 1) + (n - 2) + \cdots + 1.$$

Catatan

Selection sort dan bubble sort sama-sama sederhana dan cenderung brute-force: banyak pasangan kandidat tetap diperiksa. Orde worst case keduanya $O(n^2)$.

Sorting II: bubble sort sebagai pembandingan

Ide

Bandingkan pasangan yang bersebelahan. Jika urutannya salah, tukar. Setelah satu lintasan, elemen terbesar bergerak ke ujung kanan.



Banyak kerja

Worst case juga membentuk deret

$$(n - 1) + (n - 2) + \cdots + 1.$$

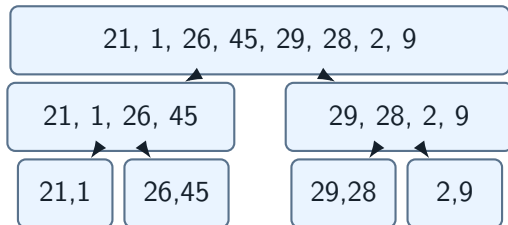
Catatan

Selection sort dan bubble sort sama-sama sederhana dan cenderung brute-force: banyak pasangan kandidat tetap diperiksa. Orde worst case keduanya $O(n^2)$.

Sorting III: merge sort

Ide

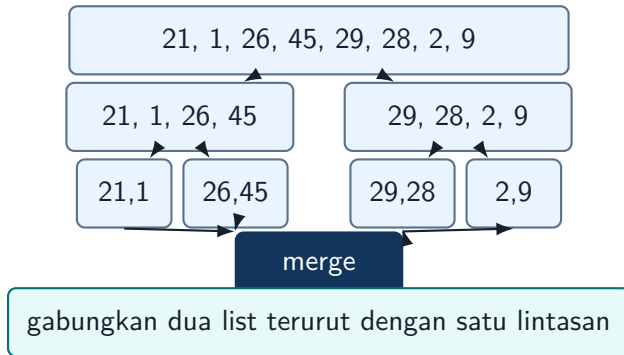
Pecah list menjadi dua bagian sampai ukuran kecil, lalu gabungkan kembali dalam keadaan terurut.



Sorting III: merge sort

Ide

Pecah list menjadi dua bagian sampai ukuran kecil, lalu gabungkan kembali dalam keadaan terurut.



Merge sort: mengapa $O(n \log n)$?

Jumlah level

Setiap pemecahan membagi ukuran menjadi dua:

$$n, n/2, n/4, \dots, 1.$$

Agar sampai ukuran 1:

$$\frac{n}{2^k} = 1 \Rightarrow k = \log_2 n.$$

Kerja per level

Pada setiap level, proses merge membaca total sekitar n elemen.

$$T(n) \approx \underbrace{n + n + \dots + n}_{\log_2 n \text{ level}} = n \log_2 n.$$

Merge sort: mengapa $O(n \log n)$?

Jumlah level

Setiap pemecahan membagi ukuran menjadi dua:

$$n, n/2, n/4, \dots, 1.$$

Agar sampai ukuran 1:

$$\frac{n}{2^k} = 1 \Rightarrow k = \log_2 n.$$

Kerja per level

Pada setiap level, proses merge membaca total sekitar n elemen.

$$T(n) \approx \underbrace{n + n + \dots + n}_{\log_2 n \text{ level}} = n \log_2 n.$$

Orde

Merge sort memiliki orde $O(n \log n)$ karena ada $\log n$ level, dan tiap level memproses total n elemen.

Algoritma	Ide utama	Worst case
Sequential search	cek satu per satu	$O(n)$
Binary search	buang separuh	$O(\log n)$
Selection sort	pilih minimum	$O(n^2)$
Bubble sort	tukar tetangga	$O(n^2)$
Merge sort	pecah dan gabung	$O(n \log n)$

Benchmark yang adil

- data input sama untuk algoritma yang dibandingkan;
- ukuran input meningkat bertahap;
- pengukuran diulang;
- laporkan median/rata-rata;
- interpretasi tetap dikaitkan dengan $T(n)$.

Waktu aktual dipengaruhi komputer, bahasa, dan implementasi; Big-O menjelaskan pola pertumbuhan saat n membesar.

- 1 Untuk list terurut $[3, 5, 6, 8, 11, 12, 14, 15, 17, 18]$, telusuri binary search untuk target 8 dan target 16.

- ① Untuk list terurut $[3, 5, 6, 8, 11, 12, 14, 15, 17, 18]$, telusuri binary search untuk target 8 dan target 16.
 - Petunjuk: pada setiap langkah, catat indeks kiri, tengah, kanan, dan nilai tengah.
- ② Lakukan tiga pass pertama selection sort pada $[11, 7, 12, 14, 19, 1, 6, 18, 8, 20]$.

- ① Untuk list terurut $[3, 5, 6, 8, 11, 12, 14, 15, 17, 18]$, telusuri binary search untuk target 8 dan target 16.
 - Petunjuk: pada setiap langkah, catat indeks kiri, tengah, kanan, dan nilai tengah.
- ② Lakukan tiga pass pertama selection sort pada $[11, 7, 12, 14, 19, 1, 6, 18, 8, 20]$.
 - Petunjuk: setelah pass ke- k , k elemen terkecil sudah berada pada posisi final.
- ③ Jelaskan mengapa insertion/selection/bubble dapat tampak cepat pada data kecil, tetapi berbeda dari merge sort ketika data membesar.

- ① Untuk list terurut $[3, 5, 6, 8, 11, 12, 14, 15, 17, 18]$, telusuri binary search untuk target 8 dan target 16.
 - Petunjuk: pada setiap langkah, catat indeks kiri, tengah, kanan, dan nilai tengah.
- ② Lakukan tiga pass pertama selection sort pada $[11, 7, 12, 14, 19, 1, 6, 18, 8, 20]$.
 - Petunjuk: setelah pass ke- k , k elemen terkecil sudah berada pada posisi final.
- ③ Jelaskan mengapa insertion/selection/bubble dapat tampak cepat pada data kecil, tetapi berbeda dari merge sort ketika data membesar.
 - Petunjuk: bandingkan bentuk n^2 dengan $n \log n$, bukan hanya satu waktu pengukuran.

Yang harus bisa dilakukan

- Menelusuri langkah searching dan sorting secara manual.
- Menentukan operasi yang dihitung sebagai kerja utama.
- Menulis $T(n)$ dari jumlah eksekusi.
- Mengambil orde dominan.

Untuk LKM M6

- Binary search: urutan nilai tengah.
- Selection sort: keadaan list setelah beberapa pass.
- Merge sort: pohon rekursi dan merge pertama.
- Benchmark: desain eksperimen, bukan sekadar angka waktu.